LINGUAGEM DE PROGRAMAÇÃO JAVA

NOSSA HISTÓRIA

A nossa história inicia com a realização do sonho de um grupo de empresários, em atender à crescente demanda de alunos para cursos de Graduação e Pós-Graduação. Com isso foi criado a nossa instituição, como entidade oferecendo serviços educacionais em nível superior.

A instituição tem por objetivo formar diplomados nas diferentes áreas de conhecimento, aptos para a inserção em setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e colaborar na sua formação contínua. Além de promover a divulgação de conhecimentos culturais, científicos e técnicos que constituem patrimônio da humanidade e comunicar o saber através do ensino, de publicação ou outras normas de comunicação.

A nossa missão é oferecer qualidade em conhecimento e cultura de forma confiável e eficiente para que o aluno tenha oportunidade de construir uma base profissional e ética. Dessa forma, conquistando o espaço de uma das instituições modelo no país na oferta de cursos, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido.

Sumário

INTRODUÇÃO 4

JAVA – CONTEXTO HISTÓRICO 5

A LINGUAGEM JAVA EM SI 7

CARACTERÍSTICAS DA LINGUAGEM 8

SIMPLICIDADE E EFICIÊNCIA DE CÓDIGO ORIENTADO A OBJETOS 8

CÓDIGO INTERPRETÁVEL E PORTÁTIL 9

SEGURANÇA 10

APLICAÇÕES DISTRIBUÍDAS E PROCESSAMENTO PARALELO 11

O COMPILADOR JAVA 12

A JVM 13

JDK 14

JAVA API 15

O AMBIENTE DE DESENVOLVIMENTO JAVA 16

ESTRUTURA DE UMA CLASSE JAVA 17

COMO CRIAR UM BOM CÓDIGO EM JAVA 18

PADRONIZAÇÃO 22

PORQUE OS DESENVOLVEDORES DE SOFTWARE ESCOLHEM JAVA?.. 23

JAVASCRIPT 24

CONSIDERAÇÕES FINAIS 25

REFERÊNCIAS 26

INTRODUÇÃO

Originalmente desenvolvida por uma equipe de desenvolvedores liderada por James Gosling na Sun Microsystems (atualmente de propriedade da Oracle) e lançada em 1995, o Java é uma linguagem de programação orientada a objetos que atualmente faz parte do núcleo da Plataforma Java.

A tecnologia Java é usada para desenvolver aplicativos para uma ampla variedade de ambientes, de dispositivos consumidores a sistemas corporativos heterogêneos.

Java é uma linguagem computacional completa, adequada para o desenvolvimento de aplicações baseadas na rede Internet, redes fechadas ou ainda programas stand-alone [CAM96].

Uma das principais vantagens da linguagem é a portabilidade, ou seja, ela é capaz de ser executada em várias plataformas como Mac, Unix e Windows, além daquela que foi criada.

O código-fonte de arquivos criados com java terminam com a extensão .java. Já os arquivos compilados (bytecodes) estão salvos com extensão .class

Para usar o Java em alguma plataforma é preciso instalar o Java SE Development Kit (JDK), que é o compilador JAVA. Para programar em java pode-se utilizar uma ferramenta para Ambiente Integrado de Desenvolvimento (IDE) utilizada para desenvolvimento de um software como Eclipse ou Netbeans. Essa última ferramenta dará ao programador a possibilidade de escrever, compilar, instalar programas, etc. Lembre-se que deverá ter um sistema operacional compatível com Java.

JAVA – CONTEXTO HISTÓRICO

Em 1991, na Sun Microsystems, foi iniciado o Green Project, o berço do Java, uma linguagem de programação orientada a objetos. Os mentores do projeto eram Patrick Naughton, Mike Sheridan, e James Gosling. Eles acreditavam que, eventualmente, haveria uma convergência dos computadores com os equipamentos e eletrodomésticos frequentemente usados pelas pessoas no seu dia-a-dia.

Para provar a viabilidade desta ideia, 13 pessoas trabalharam arduamente durante 18 meses. No verão de 1992 eles emergiram de um escritório de Sand Hill Road, no Menlo Park, com uma demonstração funcional da ideia inicial. O protótipo se chamava *7 (lê-se “Star Seven”), um controle remoto com uma interface gráfica touchscreen. Para o *7, foi criado um mascote, hoje amplamente conhecido no mundo Java, o Duke. O trabalho do Duke no *7 era ser um guia virtual ajudando e ensinando o usuário a utilizar o equipamento. O *7 tinha a habilidade de controlar diversos dispositivos e aplicações. James Gosling especificou uma nova linguagem de programação para o *7. Gosling decidiu batizá-la de “Oak”, que quer dizer *carvalho*, uma árvore que ele podia observar quando olhava através da sua janela.

O próximo passo era encontrar um mercado para o starseven. A equipe achava que uma boa ideia seria controlar televisões e vídeo por demanda com o equipamento. Eles construíram uma demonstração chamada de *MovieWood*, mas infelizmente era muito cedo para que o vídeo por demanda bem como as empresas de TV a cabo pudessem viabilizar o negócio. A ideia que o *7 tentava vender, hoje já é realidade em programas interativos e também na televisão digital. Permitir ao telespectador interagir com a emissora e com a programação em uma grande rede de cabos, era algo muito visionário e estava muito longe

do que as empresas de TV a cabo tinham capacidade de entender e comprar. A ideia certa, na época errada.

Entretanto, o estouro da internet aconteceu e rapidamente uma grande rede interativa estava se estabelecendo. Era este tipo de rede interativa que a equipe do *7 estava tentando vender para as empresas de TV a cabo. E, da noite para o dia, não era mais necessário construir a infraestrutura para a rede, ela simplesmente estava lá. Gosling foi incumbido de adaptar o Oak para a internet e em janeiro 1995 foi lançada uma nova versão do Oak que foi rebatizada para *Java* — diz-se que inspirado no café que o time de desenvolvimento consumia, oriundo da ilha de de Java, e que também está presente na logomarca *Java*.

A tecnologia Java tinha sido projetada para se mover por meio das redes de dispositivos heterogêneos, redes como a internet. Agora aplicações poderiam ser executadas dentro dos navegadores nos Applets Java e tudo seria disponibilizado pela internet instantaneamente. Foi o estático HTML dos navegadores que promoveu a rápida disseminação da dinâmica tecnologia Java. A velocidade dos acontecimentos seguintes foi assustadora, o número de usuários cresceu rapidamente, grandes fornecedores de tecnologia, como a IBM anunciaram suporte para a tecnologia Java.

Desde seu lançamento, em maio de 1995, a plataforma Java foi adotada mais rapidamente do que qualquer outra linguagem de programação na história da computação. Em 2004 Java atingiu a marca de 3 milhões de desenvolvedores em todo mundo. Java continuou crescendo e hoje é uma referência no mercado de desenvolvimento de software. Java tornou-se popular pelo seu uso na internet e hoje possui seu ambiente de execução presente em navegadores, mainframes, sistemas operacionais, celulares, palmtops, cartões inteligentes etc.

A LINGUAGEM JAVA EM SI

Como qualquer linguagem de programação, a linguagem Java tem sua própria estrutura, regras de sintaxe e paradigma de programação. O paradigma de programação da linguagem Java baseia-se no conceito de OOP, que os recursos da linguagem suportam.

A linguagem Java deriva da linguagem C, portanto suas regras de sintaxe assemelham-se às regras de C. Por exemplo, os blocos de códigos são modularizados em métodos e delimitados por chaves (`{` e `}`) e variáveis são declaradas antes que sejam usadas.

Estruturalmente, a linguagem Java começa com *pacotes*. Um pacote é o mecanismo de namespace da linguagem Java. Dentro dos pacotes estão as classes e dentro das classes estão métodos, variáveis, constantes e mais. Neste tutorial você aprende sobre as partes da linguagem Java.

Abaixo segue uma ilustração do ícone do java

(Fonte: https://www.infoescola.com/informatica/historia-do-java/)

CARACTERÍSTICAS DA LINGUAGEM

A sintaxe utilizada deriva do C++ porém com um modelo mais simples. Como sua principal característica, todo o código é escrito dentro de uma classe e tudo é um objeto, com exceção dos tipos intrínsecos (números reais e ordinais, valores booleanos e caracteres) que não são classes por questões da funcionalidade exercida.

No momento de seu desenvolvimento, os objetivos principais desejados para esta linguagem foram que ela deveria ser simples, orientada a objetos e de fácil aprendizagem não somente para programadores experientes.

Além disto, a linguagem Java deveria possuir arquitetura neutra e portável, de forma que pudesse ser utilizada em diversos Sistemas operacionais, ter alta performance, apresentar segurança e solidez e ser uma linguagem interpretada com suporte a threads e dinâmica. As aplicações em Java normalmente podem ser executadas em qualquer plataforma que possua a Java Virtual Machine (JVM) instalada, independente da arquitetura do computador.

O Java utiliza o “Garbage Collector” para gerenciar a memória referente ao ciclo de vida dos objetos e sua permanência nela. O programador determina quando os objetos são criados e o Java Runtime é responsável pela retirada do objeto da memória quando ele não estiver mais em uso, evitando que este processo seja feito manualmente como nas linguagens de programação estruturada.

SIMPLICIDADE E EFICIÊNCIA DE CÓDIGO ORIENTADO A OBJETOS

Java é uma linguagem simples, de fácil aprendizado ou migração, pois possui um reduzido número de construções. A diminuição das construções mais suscetíveis a erros de programação, tais como ponteiros e gerenciamento de memória via código de programação também faz com que a programação em Java seja mais eficiente.

Contém um conjunto de bibliotecas que fornecem grande parte da funcionalidade básica da linguagem, incluindo rotinas de acesso à rede e criação de interface gráfica. Baseada no paradigma da Orientação a Objetos - encapsulamento em um bloco de software dos dados (variáveis) e métodos de manipulação desses dados - a linguagem permite a modularização das aplicações, reuso e manutenção simples do código já implementado.

CÓDIGO INTERPRETÁVEL E PORTÁTIL

As diversas linguagens de programação podem ser tanto compiladas como interpretadas. Quando se utiliza uma linguagem compilada, é necessário executar um programa para traduzir os arquivos fonte, legíveis em linguagem de alto nível, em código executável. As linguagens compiladas têm a vantagem de produzir código de alta performance, o qual está ajustado para o funcionamento em um tipo específico de processador ou arquitetura de processador.

Aplicativos compilados, chamados de código binário, só podem rodar no tipo de computador para o qual foram compilados, uma vez que esses aplicativos consistem, na realidade, em instruções em linguagem de máquina, entendidas e executadas pelo microprocessador.

As linguagens interpretadas só existem em código fonte. Quando em execução, um programa chamado interpretador toma o código fonte e executa as ações indicadas pelos comandos no arquivo. O interpretador é, na realidade, o único aplicativo que está sendo executado.

Entre os benefícios das linguagens interpretadas está o fato dos programas interpretados poderem rodar em uma variedade de plataformas diferentes, pois estes só existem em código fonte. Além disso são mais fáceis de depurar.

A linguagem Java é tanto compilada como interpretada. Após escrever um programa em Java, utilizando um editor de textos qualquer, salva-se o programa

como código fonte. A seguir, pode-se compilar essa fonte, a fim de produzir um tipo de arquivo binário chamado de arquivo de classe.

Esses arquivos não são executados diretamente pois eles não contêm instruções que são entendidas diretamente pelos processadores atualmente disponíveis no mercado. Os programas Java são compilados em um formato intermediário chamado bytecodes.

Assim, esses programas podem ser executados em qualquer sistema através de um interpretador Java (runtime environment). Com isso, o código precisa ser escrito e compilado apenas uma vez, pois os bytecodes gerados serão executados da mesma forma em qualquer plataforma de hardware e software.

A Sun Microsystems atualmente está trabalhando em compiladores específicos para uma plataforma de hardware. Se o hardware onde o código Java será executado é conhecido, pode-se melhorar a performance do aplicativo. Em contrapartida, não será mais possível rodar esse código em outras plataformas.

Além disso, processadores que executem os bytecodes Java sem a necessidade de um interpretador também estão sendo colocados no mercado pela Sun.

SEGURANÇA

Por ter seu projeto voltado para a simplicidade de código, as possibilidades de erro de programação em Java são reduzidas. Apesar disso, a linguagem traz outros recursos para tornar seu código ainda mais eficiente. O processo de compilação - geração de bytecodes - é projetado para a detecção prévia dos possíveis erros, evitando que os erros se manifestem em tempo de execução. O uso de código para tratamento de exceções · exception handling - permite manter a consistência da aplicação no caso de erros.

Além de diminuir as possibilidades de erro de programação, a linguagem tem um esquema de segurança para garantir a integridade de código - principalmente no caso do código originário de rede insegura.

Esses recursos de segurança são notáveis principalmente dentro do ambiente do interpretador [GON97]. Após baixar um applet da rede, o interpretador faz uma verificação do código, buscando alterações intencionais ou não. A seguir, o interpretador determina o layout de memória para execução. Em outras palavras, não é possível acessar informações diretamente da memória ou inserir código estranho ao código original. Além disso, um programa em Java não pode acessar o sistema de arquivos, salvo nos casos previstos pelo cliente: diferente conjunto de permissões de acordo com a origem das aplicações [MCG97].

APLICAÇÕES DISTRIBUÍDAS E PROCESSAMENTO PARALELO

Com a larga utilização dos recursos da rede Internet, a linguagem teve seu projeto voltado para as aplicações em rede. Assim, a linguagem traz classes para o suporte a vários níveis de conectividade: acesso a URLs (padrão Internet), uso de conexões em sockets, criação de protocolos, criação de clientes e servidores.

A introdução desses conceitos se deu de forma simples para o programador, pois permite a ele o acesso às informações da rede com a mesma facilidade do acesso aos arquivos locais.

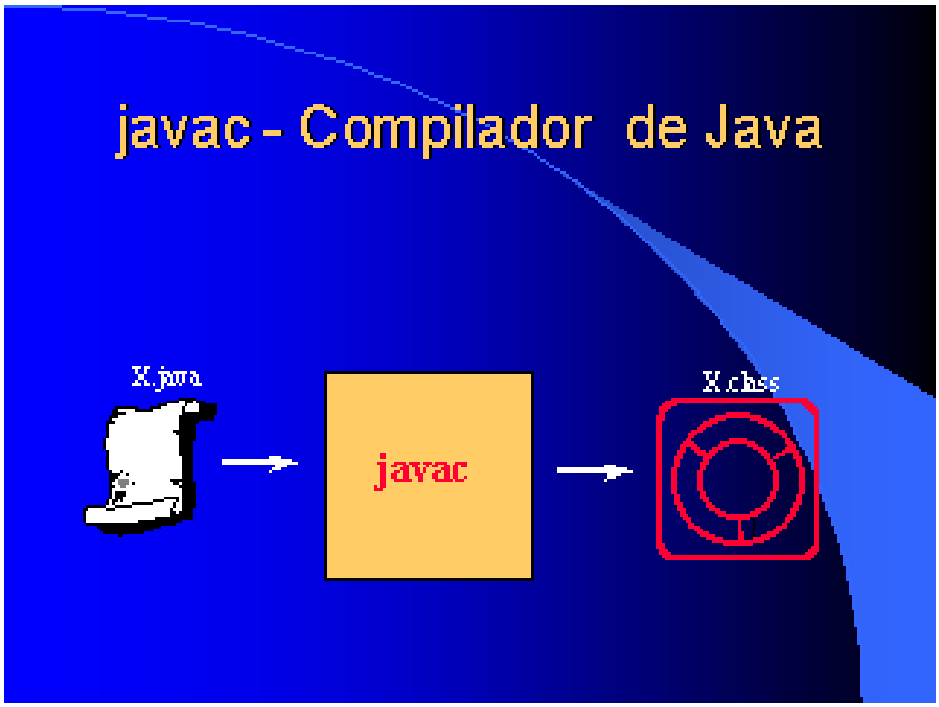
Para permitir uma melhor performance de execução, mesmo em tarefas de maior complexidade, a linguagem permite a programação de threads - processos de execução de tarefas que ocorrem simultaneamente. A linguagem traz também mecanismos para sincronização, ativação e desativação parametrizada desses processos.

O COMPILADOR JAVA

Quando você programa na plataforma Java, escreve seu código-fonte em arquivos .java e depois os compila. O compilador verifica seu código nas regras de sintaxe da linguagem e depois grava *bytecode* em arquivos .class. Bytecode é um conjunto de instruções destinadas a executar em uma Java virtual machine (JVM).

Ao incluir esse nível de abstração, o compilador Java difere-se de outros compiladores de linguagem, que escrevem instruções adequadas para o chipset de CPU no qual o programa é executado.

Segue abaixo a ilustração do compilador Java



(Fonte: https://www.cin.ufpe.br/~java/comdex96/sld045.htm)

A JVM

No tempo de execução, a JVM lê e interpreta arquivos .class e executa as instruções do programa na plataforma de hardware nativa para qual a JVM foi escrita. A JVM interpreta o bytecode como uma CPU interpretaria instruções de linguagem assembly. A diferença é que a JVM é uma parte do software escrita especificamente para uma determinada plataforma.

A JVM é o núcleo do princípio “gravação única, execução em qualquer local” da linguagem Java.

Seu código pode executar em qualquer chipset para o qual a implementação da JVM adequada está disponível. JVMs estão disponíveis para principais plataformas, como Linux e Windows, e subconjuntos de linguagem Java foram implementados nas JVMs para telefones celulares e chips hobbyist.

JDK

O JDK, abreviação para Java Development Kit, é um conjunto de utilitários cuja a finalidade é a permissão para criação de jogos e programas para a plataforma Java. Este pacote é disponibilizado pela Oracle, e nele vem todo o ambiente necessário para a criação e execução dos aplicativos java.

O Java JDK é composto pelo compilador e pelas bibliotecas (API's) necessárias para criação de programas em Java e ferramentas úteis para o desenvolvimento e para testes dos programas escritos por esta linguagem de programação. Além disso, uma Máquina Virtual Java é adicionada ao sistema operacional, no caso de ainda não ter uma instalada no computador.

O JDK dispõe de um arquivo executável que faz todo o trabalho de instalação e configuração do ambiente, o que facilita ainda mais a execução de qualquer aplicação Java e criação de novos programas de forma mais simplificada e sem muitos esforços.

JAVA API

O conjunto de bibliotecas inclusas no JDK é conhecido como Java API (Interface de Programação de Aplicações). Nessas bibliotecas estão um conjunto grande de classes, organizadas em pacotes. Cada um desses pacotes traz classes com funcionalidade básica e vital para um determinado ramo de programação Java.

A API do Java é composta por dois tipos de recursos distintos: classes e interfaces. Ela é como um bloco dividido em duas grandes partes, são centenas de Interfaces e milhares de classes que acompanham o kit de desenvolvimento do Java e que podem ser empregadas para a realização de diversos tipos de tarefas durante a construção de um programa.

Na API do Java, não existem métodos ou quaisquer tipos de dados desgarrados. O seu caráter fortemente orientado a objetos obriga a declaração de todos os métodos dentro de uma classe ou de uma interface. Do mesmo modo, não é possível declarar uma variável ou constante fora do escopo de uma classe ou de uma interface. A representação de dados deve ser feita, necessariamente, através de atributos que pertencem a uma classe ou interface específica.

As classes formam uma hierarquia dentro da API do Java, e a classe java.lang.Object é a raiz dessa hierarquia. Direta ou indiretamente, toda classe tem Object como a sua superclasse, assim, todos os métodos por ela implementados são herdados por todas as demais classes.

O AMBIENTE DE DESENVOLVIMENTO JAVA

O JDK inclui um conjunto de ferramentas de linha de comandos para compilar e executar seu código Java, incluindo uma cópia completa do JRE. Embora seja possível usar essas ferramentas para desenvolver seus aplicativos, a maioria dos desenvolvedores apreciam a funcionalidade adicional, o gerenciamento de tarefas e a interface visual de um IDE.

Eclipse é um IDE de software livre popular para desenvolvimento Java. O Eclipse manipula tarefas básicas, como a compilação e depuração de códigos, portanto, você pode focar na escrita e teste de códigos. Além disso, é possível usar o Eclipse para organizar seus arquivos de código-fonte em projetos, compilar e testar esses projetos e armazenar arquivos de projetos em qualquer número de repositórios de origem. É necessário ter um JDK instalado para usar Eclipse para desenvolvimento Java. Se você fizer o download de um dos pacotes configurá-veis Eclipse, ele já virá com o JDK.

ESTRUTURA DE UMA CLASSE JAVA

Uma classe é um blueprint para uma entidade discreta (objeto) que contém atributos e comportamentos. A classe define a estrutura básica do objeto e, no tempo de execução, seu aplicativo cria uma instância do objeto. Um objeto tem um limite e um estado nítidos e pode fazer ações quando corretamente solicitado. Cada linguagem orientada a objetos possui regras sobre como definir uma classe.

Definição de classe contém vários tipos de construções, que eu diferenciei com a formatação de fonte. As construções mostradas em negrito (que você localizará na lista de Palavras reservadas) são literais; em qualquer definição de objeto, elas devem ser exatamente o que estão na listagem. Os nomes fornecidos para outras construções descrevem os conceitos que eles representam. Eu expliquei todas as construções detalhadamente no restante desta seção.

Observação: Na Definição de classe e em alguns outros exemplos de códigos nesta seção, colchetes indicam que as construções dentro deles não são requeridas. Os colchetes (diferente de `{` e `}`) não fazem parte da sintaxe Java.

Já tivemos conhecimento suficiente na sintaxe Java para escrever programas Java básicos, o que significa que a primeira metade deste tutorial está prestes a concluir. Esta seção final expõe algumas das melhores práticas que podem ajudá-lo a escrever um código Java mais claro, com mais possibilidade de manutenção.

Mantenha classes pequenas

Por exemplo: a classe `Person` possuirá 150 linhas de código. Esta é uma classe pequena. Não é incomum ver classes com 50 ou 100 métodos e milhares de linhas de origem (ou mais). O principal ponto para métodos é manter apenas aquilo que é necessário. Se você precisar de diversos métodos auxiliares que fazem essencialmente a mesma coisa, mas obtêm parâmetros diferentes (como o método `printAudit()`), essa é uma boa escolha. Apenas certifique-se de limitar a lista de métodos ao que você precisa, e não mais.

No geral, as classes representam alguma entidade conceitual em seu aplicativo e seus tamanhos devem refletir apenas a funcionalidade para fazer o que essa entidade necessita. Elas devem permanecer amplamente focadas em efetuar um pequeno número de tarefas e fazê-las bem.

Nomeie métodos cuidadosamente

Um bom padrão de codificação quando se trata de nomes de métodos é o padrão de nomes de métodos *revelação de intenção*. Esse padrão é mais fácil de entender com um simples exemplo. Qual dos nomes de métodos a seguir é mais fácil de decifrar numa visualização rápida?

```
a()
computeInterest()
```

A resposta deveria ser óbvia, ainda que, por algum motivo, programadores tenham uma tendência a fornecer nomes de métodos (e variáveis, neste caso) pequenos e abreviados. Certamente, um nome ridiculamente longo pode ser inconveniente, mas um nome que transmita o que um método faz não precisa ser ridiculamente longo. Seis meses após escrever alguns códigos, talvez você não se lembre o que quis fazer com um método chamado `compInt()`, mas é óbvio que um método chamado `computeInterest()`, provavelmente chamará sua atenção.

Mantenha métodos pequenos

Métodos pequenos são tão preferíveis quanto classes pequenas, e por motivos semelhantes. Um padrão que eu tento seguir é manter o tamanho de um método para *uma página*, pois eu o vejo em minha tela. Isso torna minhas classes de aplicativo mais fáceis de manter.

Se um método passar de uma página, o correto é refatorar. Refatorar é mudar o design do código existente sem mudar seus resultados. O Eclipse tem um conjunto maravilhoso de ferramentas de refatoração. Geralmente, um método longo contém subgrupos de funcionalidades agrupadas. Pegue essa funcionalidade e mova-a para outro método (nomeando-a de acordo) e a transmita nos parâmetros, conforme necessário.

Limite cada método a uma única tarefa. Eu acredito que um método que faça apenas uma tarefa bem feita geralmente não possui mais do que cerca de 30 linhas de código.

Use comentários

Use comentários. As pessoas que o usarão depois (ou até mesmo você, seis meses depois) agradecerão. Talvez você conheça aquele velho ditado Um código bem escrito é, por si só, uma documentação; então quem precisa de comentários? Eu darei duas razões para mostrar porque esse ditado é falso:
A maioria dos códigos não é bem escrita.

Provavelmente seu código não estará tão bem escrito como você pensa estar. Portanto, comente seu código. Ponto.

Use um estilo consistente

O estilo de codificação é uma questão de preferência pessoal, mas eu o aconselho a usar a sintaxe Java de chaves:

```
public static void main(String[] args) {
}
```

Não use este estilo:

```
public static void main(String[] args)
{
}
```

Ou este:

```
public static void main(String[] args)
  {
  }
```

Por quê? Bem, isso é padrão, portanto a maioria dos códigos com os quais você se depara (como em código que não escreveu, mas pode ser pago para manter) provavelmente será escrita dessa forma. Dito isso, o Eclipse *permite* definir estilos de códigos e formatar seu código do modo que desejar. A principal questão é que você escolhe um estilo e adere a ele.

Use criação de log integrada

Antes que Java 1.4 apresentasse a criação de log integrada, a forma canônica de descobrir o que seu programa estava fazendo era fazer uma chamada do sistema, como a seguir:

```
public void someMethod() {
  // Do some stuff...
  // Now tell all about it
  System.out.println("Telling you all about it:");
```

```
  // Etc...
}
```

Mostrar mais

O recurso de criação de log integrado da linguagem Java (retorne à consulta a "Sua primeira classe Java") é uma melhor alternativa. Eu nunca uso `System.out.println()` em meu código e sugiro que você não o use também. Outra alternativa é a biblioteca de substituição log4j comumente usada, parte do projeto Apache umbrella.

PADRONIZAÇÃO

Em 1997 a Sun Microsystems tentou submeter a linguagem a padronização pelos órgãos ISO/IEC e ECMA, mas acabou desistindo.[3][4][5] Java ainda é um padrão de fato, que é controlada através da JCP Java Community Process.[6] Em 13 de novembro de 2006, a Sun lançou a maior parte do Java como Software Livre sob os termos da GNU General Public License (GPL). Em 8 de maio de 2007 a Sun finalizou o processo, tornando praticamente todo o código Java como software de código aberto, menos uma pequena porção da qual a Sun não possui copyright.

PORQUE OS DESENVOLVEDORES DE SOFTWARE ESCOLHEM JAVA?

O Java foi testado, refinado, estendido e comprovado por uma comunidade dedicada de desenvolvedores, arquitetos e entusiastas do Java. O Java foi projetado para permitir o desenvolvimento de aplicações portáteis de alto desempenho para a mais ampla variedade possível de plataformas de computação. Ao disponibilizar aplicações entre ambientes heterogêneos, as empresas podem fornecer mais serviços e aumentar a produtividade, a comunicação e a colaboração do usuário final — além de reduzir drasticamente o custo de propriedade das aplicações da empresa e do consumidor. O Java tornou-se inestimável para os desenvolvedores, permitindo que eles:

- Gravem software em uma plataforma e o executem virtualmente em qualquer outra plataforma
- Criem programas que podem ser executados dentro em um web browser e acessem web services disponíveis
- Desenvolvam aplicações do servidor para fóruns on-line, armazenamentos, pesquisas, processamento de forms HTML e mais
- Combinem aplicações ou serviços usando a linguagem Java para criar aplicações ou serviços altamente personalizáveis.
- Crie aplicações potentes e eficientes para telefones celulares, processadores remotos, microcontroladores, módulos sem fio, sensores, gateways, produtos de consumo e praticamente qualquer outro dispositivo eletrônico

JAVASCRIPT

A criação do JavaScript foi influenciada por várias linguagens de programação e ela foi desenvolvida para parecer-se com a linguagem Java, porém em um formato mais simples que facilitasse seu uso não apenas para programadores. O principal objetivo do JavaScript é a construção de páginas da internet mais dinâmicas.

O JavaScript é uma linguagem de scripts utilizada para acessar objetos dentro de outras aplicações. Ela é utilizada em milhares de páginas da internet para acrescentar funcionalidades, validação de formulários, detectar navegadores, entre diversas outras aplicações.

Alguns exemplos simples de sua utilização são: abertura de uma nova janela com controle programático sobre o tamanho, posição e atributos dela; validação de dados inseridos em formulários web de forma a garantir que eles sejam aceitos antes de serem submetidos ao servidor; alteração de imagens conforme o mouse passe sobre elas.

CONSIDERAÇÕES FINAIS

Atualmente, já existem compiladores capazes de traduzir bytecodes para instruções de máquina nativas, como o Just In Time compiler (ou JIT), tornando os programas ainda mais rápidos. Este compilador requer uma versão específica para cada plataforma onde se pretende que o programa Java seja executado.

Em contrapartida à maior velocidade de execução está também uma maior necessidade de memória, pois os bytecodes compilados, em geral, ficam três vezes maiores do que o original. Uma alternativa bem mais interessante, e talvez muito mais viável, é a implementação da JVM em hardware na forma de uma placa ou microchip.

A primeira iniciativa neste sentido é da Sun Microelectronics, que está produzindo os chips picoJava ITM, microJavaTM e UltraJavaTM. Estes são capazes executar diretamente bytecodes, acelerando em milhares de vezes a velocidade de execução. Isto permitirá o desenvolvimento viável de aplicativos cada vez mais complexos, abrangentes e funcionais.

Espera-se que estas soluções sejam brevemente empregadas na fabricação de telefones celulares, pagers, jogos, organizadores pessoais digitais, impressoras e eletrodomésticos de consumo, além aplicações mais sérias como estações de trabalho dotadas do sistema operacional JavaOSTM. Trata-se certamente do futuro das soluções para aplicações de rede

Quando você for se aventurar no mundo da programação, é bom já ter uma prévia noção da importância e da capacidade da linguagem Java.

REFERÊNCIAS

https://www.tecmundo.com.br/programacao/2710-o-que-e-java-.htm

https://www.java.com/pt-BR/about/

https://www.programador.com.br/linguagens-de-programacao/linguagem-java.html

https://pt.wikipedia.org/wiki/Java_(linguagem_de_programa%C3%A7%C3%A3o)

https://www.sct.ce.gov.br/utd/java-com-banco-de-dados/#:~:text=O%20que%20%C3%A9,s%C3%A3o%20compiladas%20para%20c%C3%B3digo%20nativo.

https://developer.ibm.com/br/tutorials/j-introtojava1/

https://www.cin.ufpe.br/~arfs/introjava.pdf

https://www.devmedia.com.br/introducao-ao-java-jdk/28896

https://tableless.com.br/java-introducao-a-sua-api/

https://www.dm.ufscar.br/~waldeck/curso/java/introd.html